Homework 3: root finding and optimization

Submission

Due: Oct 12, 2026

Submit the following three files in your directory:

1. `lastname_firstname_hw03.ipynb`: one executed Jupyter notebook containing all code and markdown text;
2. `lastname_firstname_hw03.pdf`: a PDF compiled or exported from the executed notebook; and
3. `requirements.txt`: a short list of the packages needed to run the notebook, with version numbers.

Your notebook must run from top to bottom in a fresh session without manual edits. Use the supplied random seeds. Report requested numerical results with at least three significant digits. Keep written explanations concise, but explain the numerical behavior rather than reporting outputs alone.

Question 1: Solving a likelihood equation (30 points)

Consider a logistic model with a known offset z_i and an unknown intercept θ:

$$\Pr(Y_i = 1) = \frac{1}{1 + \exp\{-(\theta + z_i)\}}.$$

The derivative of the negative log-likelihood is

$$g(\theta) = \sum_{i=1}^{n} \{p_i(\theta) - y_i\}, \qquad g'(\theta) = \sum_{i=1}^{n} p_i(\theta)\{1 - p_i(\theta)\}.$$

Generate the data and define the functions using:

```
import numpy as np
import matplotlib.pyplot as plt
from scipy.special import expit

rng = np.random.default_rng(123)
n = 2000
offset = rng.normal(scale=1.5, size=n)
theta_true = -0.7
y = rng.binomial(1, expit(theta_true + offset))

def g(theta):
    return np.sum(expit(theta + offset) - y)

def g_prime(theta):
    p = expit(theta + offset)
    return np.sum(p * (1 - p))
```

1. Evaluate $g(\theta)$ on 200 equally spaced points from -5 to 5. Plot the curve with a horizontal line at zero. Report $g(-5)$ and $g(5)$. Explain why these results suggest that a unique root lies in $[-5, 5]$.

2. Implement the bisection method on $[-5, 5]$. Stop when either $|g(\theta_k)| < 10^{-8}$ or the bracket width is below 10^{-8}. Report the estimated root, final residual, number of iterations, and final bracket width. Do not call a prewritten root-finding function for this part.

3. Implement Newton's method for at most 20 iterations. Stop when $|g(\theta_k)| < 10^{-8}$, or declare failure if the derivative is below 10^{-12} or an iterate becomes nonfinite. Run it from $\theta_0 = 0$ and $\theta_0 = 3$. Report the iterate sequence for each initialization and explain why Newton succeeds from one starting point but fails from the other. Compare this behavior with bisection.

Question 2: Step size and curvature in optimization (35 points)

Consider the positive-definite quadratic

$$f(x) = \frac{1}{2}x^\top Ax - b^\top x, \qquad \nabla f(x) = Ax - b.$$

Use the following setup:

```
import numpy as np

angle = np.pi / 6
R = np.array([
    [np.cos(angle), -np.sin(angle)],
    [np.sin(angle),  np.cos(angle)],
])
A = R @ np.diag([1.0, 100.0]) @ R.T
b = np.array([1.0, -1.0])
x0 = np.array([3.0, -2.0])

def objective(x):
    return 0.5 * x @ A @ x - b @ x

def gradient(x):
    return A @ x - b
```

1. Compute the eigenvalues and condition number of A, and obtain the exact minimizer x^* by solving $Ax^* = b$. Use $\lambda_{\max}(A)$ to state the interval of fixed step sizes for which gradient descent converges on this quadratic.

2. Starting from `x0`, run 100 gradient-descent iterations using each step size

$$\alpha \in \{0.005,\ 0.015,\ 0.021\}.$$

   Plot $f(x_k) - f(x^*)$ against iteration on a logarithmic vertical scale. Report the final objective gap and gradient norm for each step size. Explain the three trajectories using the convergence interval from Part 1.

3. Implement gradient descent with an Armijo backtracking line search using initial step size 1, contraction factor 0.5, and Armijo constant 10^{-4}. Stop when $\|\nabla f(x_k)\|_2 < 10^{-6}$ or after 2,000 iterations. Report the final point, objective gap, gradient norm, iteration count, and number of objective evaluations.

Question 3: Mini-batch logistic regression (35 points)

For binary responses $y_i \in \{0, 1\}$ and predictors $x_i \in \mathbb{R}^p$, consider the regularized training objective

$$f(\beta) = \frac{1}{n}\sum_{i=1}^{n}\left[\log\{1 + \exp(x_i^\top \beta)\} - y_i x_i^\top \beta\right] + \frac{\lambda}{2}\|\beta\|_2^2.$$

Use `np.logaddexp` when evaluating the logistic loss. Generate the data and define the helper functions using:

```python
import numpy as np
from scipy.special import expit
from time import perf_counter

rng = np.random.default_rng(789)
n_train, n_valid, p = 8000, 2000, 20
X = rng.normal(size=(n_train + n_valid, p))
beta_true = rng.normal(scale=0.35, size=p)
y = rng.binomial(1, expit(X @ beta_true))
X_train, X_valid = X[:n_train], X[n_train:]
y_train, y_valid = y[:n_train], y[n_train:]
lam = 1e-3

def data_loss(beta, X, y):
    eta = X @ beta
    return np.mean(np.logaddexp(0, eta) - y * eta)

def train_objective(beta):
    return data_loss(beta, X_train, y_train) + 0.5 * lam * (beta @ beta)

def batch_gradient(beta, X, y):
    return X.T @ (expit(X @ beta) - y) / len(y) + lam * beta
```

Use a mini-batch size of 64 and a budget of $50n_{\text{train}}$ processed observations. Pre-generate one shared sequence of mini-batches so the stochastic methods see the same data:

```python
batch_size = 64
epochs = 50
steps = epochs * n_train // batch_size
batch_rng = np.random.default_rng(2468)
batch_indices = batch_rng.integers(
    0, n_train, size=(steps, batch_size)
)
```

Start every method from $\beta_0 = 0$. Record the training objective, validation loss, and elapsed wall-clock time after every n_{train} processed observations.

1. Implement full-batch gradient descent with step size 1.0 and mini-batch SGD with step size 0.05. Run each method for the full observation budget. For full-batch gradient descent, one update processes n_{train} observations. For mini-batch SGD, one update processes 64 observations. Report the final training objective, validation loss, and runtime. **(15 points)**

2. Implement the following two methods using the supplied mini-batch sequence. **(20 points)**

   - **SGD with momentum:** use step size 0.05 and $\gamma = 0.9$,

   $$v_t = \gamma v_{t-1} + (1-\gamma) g_t, \qquad \beta_t = \beta_{t-1} - 0.05 v_t.$$

   - **Adam:** use step size 0.002, $\beta_1 = 0.9$, $\beta_2 = 0.999$, $\epsilon = 10^{-8}$, and the standard bias corrections for the first and second moments.

   Report the final training objective, validation loss, and runtime for both methods.

Grading guide

Criterion	Points
Correct implementation and numerical results	50
Sound analysis and interpretation	25

Criterion	Points
Clear figures, tables, and concise communication	15
Reproducibility and complete submission	10
Total	**100**

Minor differences caused by software versions and timing variability are acceptable. Numerical results without written interpretation will receive limited credit.